

Spring AI

1. Overview of Spring AI

Spring AI brings the foundational design principles of the Spring ecosystem—portability, dependency injection, and clean abstractions—into generative artificial intelligence.

Instead of hardcoding integration logic to specific vendor SDKs, Spring AI decouples your domain logic through uniform interfaces. This allows applications to switch between foundational model providers, vector databases, and document parsers with minimal configuration changes.

2. The ChatClient Abstraction

The ChatClient represents the primary conversational interface in Spring AI. It provides an ergonomic, fluent builder pattern designed to coordinate the entire lifecycle of a language model interaction.

- **Fluent Request Pipeline:** ChatClient models interactions similarly to standard Spring client utilities. It allows developers to specify user input, system instructions, temperature, and runtime options within a single continuous flow.
- **Separation of Roles:** It cleanly separates system-level instructions (which dictate model persona and constraints) from dynamic user prompts and multi-turn context.
- **Structured Output Mapping:** Beyond returning raw text responses, ChatClient natively maps unstructured model responses into strongly typed domain objects. The framework automatically injects schema definitions into the prompt and deserializes the returned output into target classes.
- **Streaming Responses:** It natively handles asynchronous, token-by-token response streaming, allowing real-time text delivery to downstream clients without waiting for complete generation.
- **Advisors Interceptor Chain:** ChatClient features an extensible middleware layer called Advisors. Advisors can inspect, enrich, or modify prompts before they reach the model, as well as handle post-processing, logging, security filtering, and context enrichment.

3. Vector Embeddings and Vector Databases

Large language models work with human language, but semantic search requires a mathematical representation of meaning. Vector embeddings bridge this gap.

- **Vector Embeddings Defined:** An embedding is a sequence of floating-point numbers representing the semantic and contextual meaning of a block of text in high-dimensional space. Words and phrases with similar concepts or intent are positioned geometrically close to one another, regardless of whether they share the exact same vocabulary.
- **The Embedding Model:** Spring AI provides a dedicated model abstraction responsible for taking text inputs and calling an embedding model to generate high-dimensional vectors.
- **Vector Stores (Vector Databases):** Traditional relational databases excel at exact-match indexing, but they cannot natively perform semantic search. Vector stores persist vectors alongside text chunks and metadata.
- **Similarity Algorithms:** When querying a vector store, the system measures distance rather than exact keyword overlap. Common mathematical operations used include:
 - **Cosine Similarity:** Measures the angle between two vectors, focusing entirely on orientation rather than magnitude.
 - **Euclidean Distance:** Measures the straight-line physical distance between points in multi-dimensional space.

- **Dot Product:** Measures directional alignment and magnitude combined.

4. Retrieval-Augmented Generation (RAG)

Foundational models possess vast general knowledge, but they lack access to real-time information, private internal data, and specialized organizational documents. Furthermore, models can hallucinate facts when operating outside their training set.

Retrieval-Augmented Generation solves this limitation by fetching relevant, factual source data dynamically and feeding it to the model as grounding context.

The Standard RAG Lifecycle

1. Document Ingestion and Preparation:

- **Extraction:** Documents (such as technical manuals, policies, or web pages) are ingested and stripped of formatting.
- **Chunking:** Large texts are split into smaller, coherent segments. Chunking is essential because models have limited input capacity, and smaller chunks preserve specific context much better than entire files.
- **Vectorization:** Each chunk is passed through an embedding model to compute its vector representation.
- **Persistence:** The text chunk, its generated vector, and any metadata (source URI, title, author) are stored in the vector database.

2. Dynamic Retrieval:

- A user submits a query.
- The query itself is converted into a vector using the exact same embedding model used during ingestion.
- The vector store searches for the most relevant document chunks based on mathematical similarity scores.
- Chunks meeting a defined relevance threshold are gathered.

3. Augmentation and Generation:

- The retrieved document chunks are stitched into the user's prompt as explicit context.
- The system prompt instructs the model to answer the question relying strictly on the injected context.
- The language model synthesizes a concise, grounded response without inventing facts.

5. Automated RAG Through Advisors

Spring AI streamlines the RAG pattern by embedding retrieval directly into the ChatClient advisor chain.

Rather than requiring manual orchestration—retrieving the query, calling the vector store, building a prompt template, and executing the call—Spring AI provides specialized Question-Answer Advisors:

- **Transparent Interception:** When a user issues a prompt, the advisor intercepts the call before it goes to the model.
- **Automated Querying:** The advisor automatically issues a similarity search against the configured vector store using the prompt text.

- **Context Assembly:** It automatically formats the top-matching documents and appends them to the prompt envelope.
- **Execution:** The enriched prompt is forwarded to the LLM, returning a verified, context-aware answer seamlessly.

6. Architectural Best Practices

- **Chunk Size Calibration:** Avoid chunks that are too large (which dilute specific facts) or too small (which lack surrounding narrative context). Chunks between a few hundred tokens with slight overlap typically yield optimal retrieval performance.
- **Similarity Threshold Filtering:** Always enforce a minimum similarity threshold during retrieval. Without a cutoff, the database will return irrelevant chunks if no good match exists, which degrades response quality.
- **Negative Prompting:** Explicitly instruct the model to declare when it lacks sufficient context to answer, preventing speculative guesses.
- **Index Types:** For production vector storage, choose indexed vector spaces (such as Hierarchical Navigable Small World graphs) to ensure low-latency lookups as datasets scale to millions of records.